

Class 06 — Student Notes

Bong Study Hub — Foundation Batch 2026

Variables & Data Types

Giving Values a Label That Can Change

These notes summarize what we covered in class. Use them to revise, not to learn the topic for the first time — the real learning happened at the keyboard, especially the moment `print(name)` and `print("name")` showed two different things. No arithmetic, no `input()`, and no `if/else` appear here — those come in later classes.

The Big Question

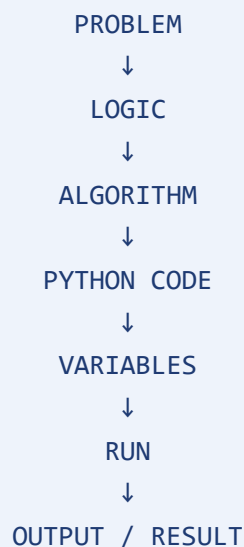
Class 05 ended with every program saying the exact same fixed thing, every single time it ran. If the text needed to change, the only option was to edit the code and run it again.

"If a program needs to remember something that changes — a score, an age, an answer — how does it hold onto it?"

Quick connection to Class 04 and Class 05

Class 04 taught us that information gets represented inside a computer as **bits** — you never see those bits directly. Class 05 gave us `print()`, quotes, and fixed text. Today connects both:

A **variable** is the human-readable label Python gives you for a piece of stored, changeable information. You write `age`. Python quietly manages the bits underneath.



By the end of today, you will have personally created, printed, and changed several variables.

1. What Is a Variable?

Imagine a small labeled box. The label says `SCORE`. Right now it holds `0`. Later, you can take out the `0` and put `10` in its place — same label, new contents. That's exactly what a variable is.

```
score = 0
print(score)
```

VARIABLE = a named place to store a value — and unlike Class 05's fixed text, its value can change later. `score` is the label. `0` is what's currently inside.

2. Assignment: The = Sign

In math class, `=` means "equals — both sides are the same." In Python, `=` means something different: **store**.

```
score = 0
print(score)
score = 10
print(score)
```

Read `score = 0` **out loud as "score gets 0,"** not "score equals 0." If `=` meant mathematical equality, `score` couldn't equal both 0 and 10. It's not an equation — it's an instruction: store this value, right now, under this name.

3. Variables in print()

This is the single easiest mistake to make in the next several classes — so let's meet it on purpose, now.

```
name = "Priya"
print(name) # Priya
print("name") # name
```

- `print(name)` — no quotes — means **"show me whatever value is stored in the variable `name`."**
- `print("name")` — with quotes — means **"show me the literal four letters, n-a-m-e."**

Quotes are the entire difference.

4. Reassignment

```
score = 0
print(score)
score = 10
print(score)
score = 25
print(score)
```

After `score = 10` runs, the `0` is **completely gone** — not hidden, not remembered anywhere. A variable holds exactly one value at a time. Each reassignment replaces whatever was there before.

5. Naming Variables

A variable name:

- Must start with a **letter or an underscore**, not a digit.
- Can contain letters, digits, and underscores — but **no spaces**.
- Is **case-sensitive** — `age` and `Age` are two different variables.
- Can't be one of Python's own reserved words, like `print`.

```
age = 18 # valid
student_name = "Rio" # valid
_temp = 5 # valid

2nd_place = "Sam" # invalid – starts with a digit
student name = "Rio" # invalid – contains a space
```

Style tip: Python style favors `snake_case` — lowercase words joined by underscores — and **descriptive names**. `student_age` beats `x`.

6. Meet the Data Types

```
age = 18
price = 19.99
name = "Priya"
is_student = True
```

Four lines, four different **kinds** of value — Python calls this a **data type**:

Variable	Value	Data type	What it means
----------	-------	-----------	---------------

age	18	int	A whole number
price	19.99	float	A number with a decimal point
name	"Priya"	str	Text, in quotes
is_student	True	bool	A yes/no fact — True or False

You can even ask Python directly:

```
print(type(age))
```

→ <class 'int'>

7. Numbers: int vs float

```
apples = 3 # int – a whole number
temperature = 36.6 # float – has a decimal point
```

`int` (integer) = a whole number, positive or negative, no decimal point. `float` = any number with a decimal point, even something like `5.0`. The decimal point is the entire test.

We're not doing any math with these numbers yet — that's next class.

8. Strings, Revisited

The Class 05 rule hasn't changed: literal text still needs quotes. What's new today: **a variable name is never in quotes**.

```
city = "Kolkata"
print(city) # Kolkata – city is a variable, no quotes needed
print("city") # city – the literal word, because of the quotes
```

9. Booleans

```
is_raining = True
is_weekend = False
print(is_raining)
```

A **boolean** holds exactly one of two values: `True` or `False` — capital letters, **no quotes**. It represents a yes/no, on/off kind of fact.

`True` and `"True"` are not the same thing. Without quotes, it's a boolean. With quotes, it's just a four-letter string.

10. Combining Text and Variables in `print()`

```
name = "Priya"
age = 18
print("My name is", name, "and I am", age, "years old.")
```

Output:

```
My name is Priya and I am 18 years old.
```

A comma inside `print()` lets you mix literal text and variables in one line. Python automatically puts a single space between each piece.

11. Connecting Back to Class 04

Remember Class 04? A computer represents information as bits — 0s and 1s, combined. You never actually see those bits when you write Python.

```
INFORMATION
  ↓
REPRESENTATION (Class 04)
  ↓
BITS (Class 04)
  ↓
VARIABLE – a human-readable label for it (today)
```

A variable is the human-readable label Python gives you for a piece of that stored representation. You write `age`. Underneath, Python and the computer are managing bits — you just never have to think about them directly.

Try It Yourself — Variable Profile Card

Build a "Variable Profile Card" about yourself, or a character you invent. You need **at least four variables**: one string, one integer, one float, and one boolean. Then use `print()` with commas to display each one in a full sentence.

```
name = "____"  
age = ____  
height = ____  
is_student = ____  
  
print("My name is", name)  
print("I am", age, "years old")  
print("My height is", height, "cm")  
print("Am I a student?", is_student)
```

Fill in your own details, run it, and then try printing one variable name in quotes on purpose — notice how the output changes, and explain why.

Quick Recap

- Class 05's fixed text couldn't change without editing and re-running the code — a **variable** solves that.
- `=` means **store**, not mathematical equality — read it as "gets."
- `print(name)` shows the stored value; `print("name")` shows the literal word — quotes are the entire difference.
- **Reassignment** completely replaces a variable's old value — nothing is kept.
- Variable names must start with a letter/underscore, contain no spaces, and are **case-sensitive**.
- The four data types from today: `int`, `float`, `str`, `bool`.
- A comma inside `print()` combines literal text and variables, with an automatic space between each piece.
- A variable is a human-readable label for information stored as bits — reconnecting straight back to Class 04.

Key Terms

Term	Meaning
Variable	A named container that stores a value — and can be given a new value later.
Assignment (=)	The act of storing a value in a variable.

Value	The actual piece of data itself.
Data type	The kind of value a variable holds.
Integer (<code>int</code>)	A whole number, positive or negative, no decimal point.
Float	A number with a decimal point.
String (<code>str</code>)	Text, written between matching quotes.
Boolean (<code>bool</code>)	One of exactly two values: <code>True</code> or <code>False</code> .
Reassignment	Giving a variable a new value, completely replacing the old one.
<code>type()</code>	A built-in tool that reports a value's data type.

Think About It

Try answering these in your own words — you don't need exact wording, just the right idea.

1. Why wasn't Class 05's fixed text enough for something like a changing score?
2. What does `=` actually do in Python?
3. What's the difference between `print(name)` and `print("name")`?
4. After you reassign a variable, what happens to its old value?
5. Name the four data types from today, with one example of each.
6. How does a variable connect back to Class 04's bits?

Final Takeaway

"Last class, your programs could only ever say the exact same thing, every single time you ran them. Today, they can hold something, change it, and tell you about it. That one idea — a labeled value that can change — is what makes a program feel alive instead of frozen."

Right now, we can only print numbers — we haven't actually done any math with them. What if a program needed to add a score, calculate a total, or compare two values? That's exactly where we pick up next class.